# DAY 5 – LAB 5.2

# Web App to Azure Queue using Managed Identity and Deployment Slots

## 1. Business Scenario

Contoso Orders has a web application where customers submit orders.

Currently, the application hosts the user interface. The next requirement is to send submitted orders to a background processing system instead of performing all processing directly inside the web application.

The organization wants to:

- Decouple order submission from order processing.
- Use Azure Queue Storage to hold submitted orders.
- Authenticate using Managed Identity instead of storage account keys.
- Test application updates in a staging environment before promoting them to production.

**Business objective:** Implement secure asynchronous messaging between the web application and Azure Storage Queue, then demonstrate controlled application deployment using deployment slots.

## 2. Architecture & Technology Overview

**Application architecture**

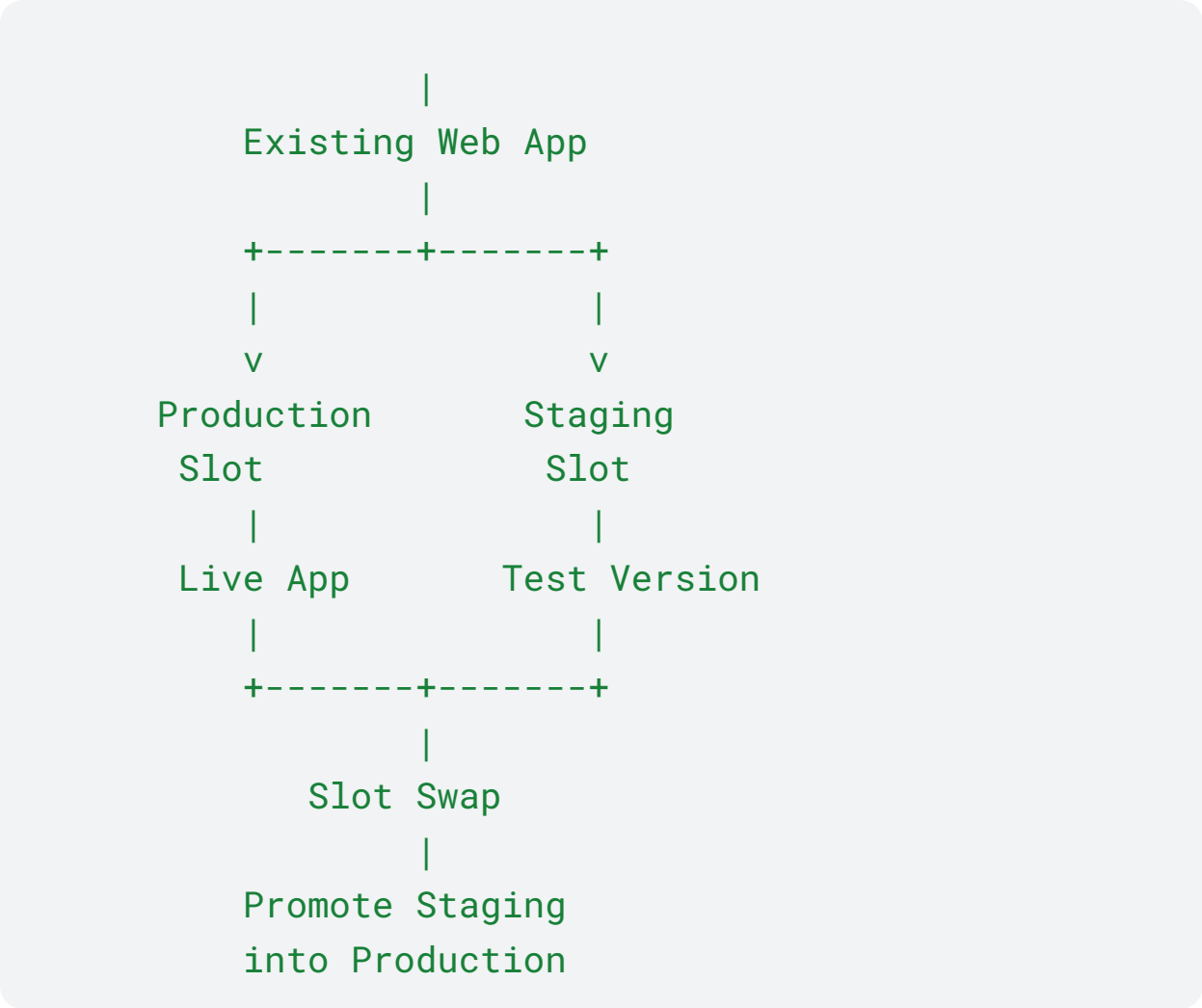
None

```
            User / Browser
                   |
                   v
          Azure App Service
          ASP.NET Core Web App
                   |
                   | Submit Order
                   v
          System-assigned
          Managed Identity
                   |
                   | Microsoft Entra ID
                   | Token authentication
                   v
          Azure Storage Account
                   |
                   v
            Queue Storage
              orders
                   |
                   v
          Lab 5.3 – Azure Function
          Background Order Processing
```

**Deployment architecture**

None

```
        Azure App Service Plan
            Standard S1
```




**Key Concepts**

| Component | Purpose |
|---|---|
| Azure Storage Account | Provides the storage infrastructure for Queue Storage. |
| Azure Queue Storage | Stores order messages for asynchronous processing. |
| Queue | Holds individual messages until a consumer processes them. |

| | |
|---|---|
| Managed Identity | Provides passwordless authentication for Azure resources. |
| Microsoft Entra ID | Issues identity tokens used for authentication. |
| Azure RBAC | Controls which operations an identity can perform. |
| Azure.Identity | Provides authentication through DefaultAzureCredential. |
| Azure.Storage.Queues | .NET SDK used to communicate with Queue Storage. |
| Deployment Slots | Provides separate application environments for testing and production releases. |
| Slot Swap | Promotes the tested staging application into production. |

**Understanding Asynchronous Messaging**

In a traditional synchronous approach, the web application may submit an order and wait for the complete processing operation to finish before responding.

This can increase response times and tightly couple the web application to downstream processing systems.

With asynchronous messaging:

1. The user submits an order.
2. The web application validates the request.
3. The application publishes an order message to Azure Queue Storage.
4. The application confirms successful submission.

5. A separate background processor handles the message later.

**Real-world use cases:**

- E-commerce order processing.
- Payment processing workflows.
- Email and notification systems.
- Document processing pipelines.
- Background report generation.

**Why Azure Queue Storage?**

It provides a managed message queue that allows application components to communicate without requiring the producer and consumer to execute at the same time.

Our web application will act as the message producer. The Azure Function introduced in Lab 5.3 will act as the consumer.

**Understanding Managed Identity**

Managed Identity provides an automatically managed identity for Azure resources.

Instead of storing a storage account key or connection string inside application code, the Web App obtains an authentication token through its managed identity.

**Real-world example:**

An enterprise application needs to upload documents to Azure Storage. Rather than storing storage credentials in configuration files, the organization enables Managed Identity and grants the application only the required storage permissions.

**Authentication flow:**

1. Enable Managed Identity on the Web App.
2. Assign the required Azure RBAC role.
3. The application requests an authentication token.
4. Microsoft Entra ID provides the token.
5. Azure Storage validates the identity and permissions.
6. The application performs the authorized operation.

**Important distinction:**

Authentication establishes who the application is. Authorization determines what that identity is allowed to do.

#### Understanding Deployment Slots

Deployment slots provide separate environments within an Azure Web App.

For example:

- Production: Current live application.
- Staging: Updated application awaiting validation.

Instead of directly replacing the production application, we deploy the updated version to staging, test it and then swap the slots.

**Real-world use case:** An organization wants to release a new order-submission feature without immediately replacing its live application. The development team deploys the update to staging, performs validation and promotes it to production after approval.

## 3. Resources and Naming

Reuse the Lab 5.1 resources. Create only the Storage Account and staging deployment slot.

| Resource | Example name | Action |
|---|---|---|
| Resource Group | Existing shared RG | Reuse |
| App Service Plan | asp-user01-d5 | Reuse |
| Web App | web-user01-orders-d5 | Reuse |
| Storage Account | stuser01ordersd5 | Create |
| Queue | orders | Create |
| Deployment Slot | staging | Create |

Replace `user01` with your assigned participant suffix.

**Storage Account naming rules:**

- Lowercase letters and numbers only.
- 3–24 characters.
- Globally unique.
- No hyphens or special characters.

## 4. Prerequisites

- Lab 5.1 Web App is deployed and running.
- Existing App Service Plan uses Standard S1.
- .NET 10 SDK and VS Code are available.
- Azure App Service extension is installed and signed in.
- Permission to create Storage Accounts and deployment slots.

- Permission to create Azure role assignments, or administrator support.
- Access to the existing shared Resource Group.

**Administrator consideration:** If participants need to inspect queue messages using Microsoft Entra authentication in Azure Portal, their user accounts may also require suitable Reader and Storage Queue Data Reader permissions. This is separate from the application's sender-only identity.

---

# 5. Lab Execution

## Phase 1 – Create the Storage Account and Queue

**Concept:** Azure Storage Account provides the underlying storage infrastructure. Queue Storage is one of its data services and allows applications to store messages for asynchronous processing.

**Why are we creating it?**

Our order application needs a reliable intermediate location where it can publish orders without waiting for the background processor to complete its work.

**Execution – Create Storage Account**

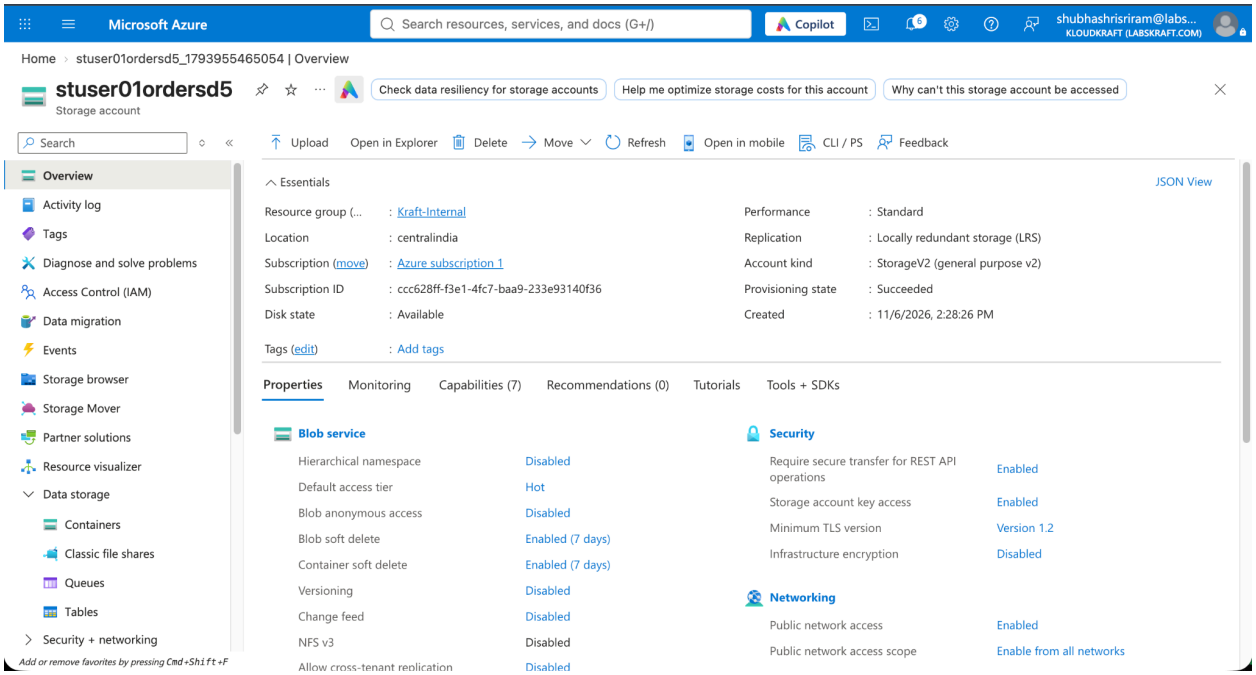
Open Azure Portal → Storage accounts → Create.

Configure:

| Setting | Value |
|---|---|
| Subscription | Training subscription |

| Resource Group | Existing shared RG |
| --- | --- |
| Storage Account Name | stuser01ordersd5 |
| Region | Same as Web App |
| Performance | Standard |
| Redundancy | Locally redundant storage (LRS) |

Keep the remaining defaults unless the training environment specifies otherwise.

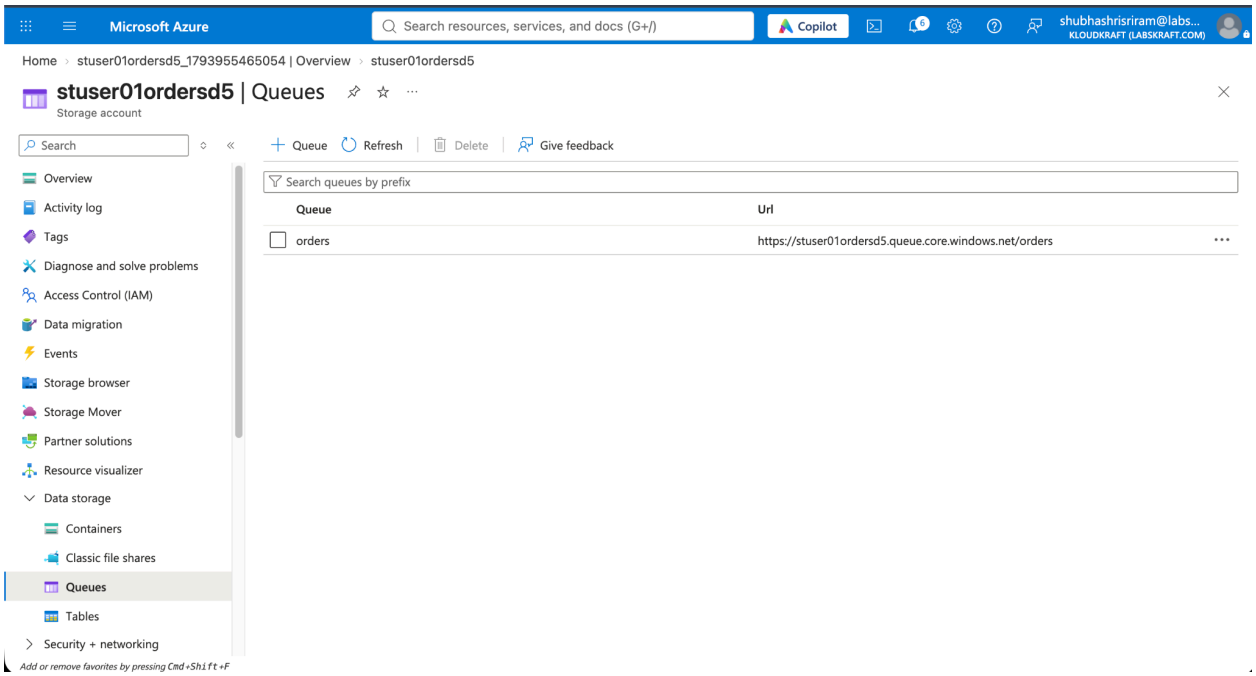



Select Review + create → Create.

## Execution – Create Queue

1. Open the newly created Storage Account.
2. Navigate to Data storage → Queues.
3. Select + Queue.
4. Enter `orders`.

5. Select OK.




### Engineering understanding

The Storage Account provides the storage endpoint, while the `orders` queue stores individual messages.

The Web App will publish messages to this queue. A separate Azure Function will consume them in Lab 5.3.

**Expected outcome:** Storage Account is provisioned and the `orders` queue is visible.

## Phase 2 – Enable Managed Identity on Production Web App

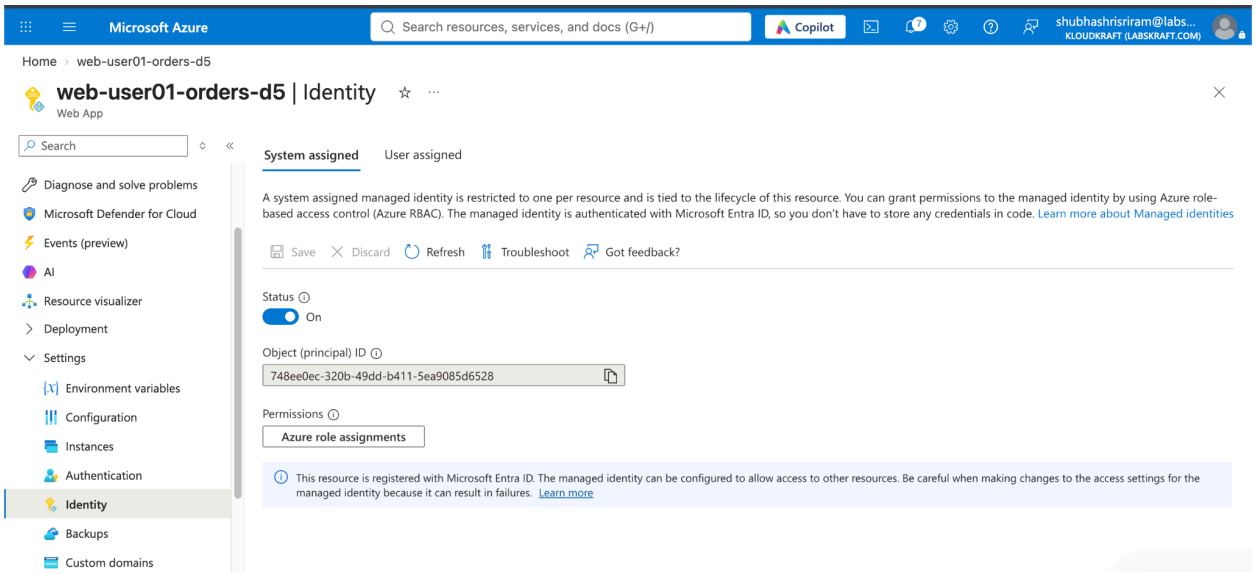
**Concept:** A system-assigned managed identity creates an identity associated with the Azure resource. Azure manages its credentials and lifecycle.

### Why are we enabling it?

Our Web App needs to authenticate to Queue Storage without storing account keys or connection strings.

## Execution

1. Open `web-user01-orders-d5`.
2. Navigate to Settings → Identity.
3. Under System assigned, change Status to On.
4. Select Save and confirm.
5. Record the displayed Object (principal) ID.




## Engineering understanding

Enabling the identity creates a corresponding service principal in Microsoft Entra ID.

The application can use this identity to request access tokens when communicating with supported Azure services.

**Important:** The staging deployment slot has a separate identity configuration. We will enable and authorize it independently.

**Expected outcome:** Production Web App has its system-assigned Managed Identity enabled.

## Phase 3 – Grant Queue Permissions to Production

### Concept – Azure RBAC

Azure Role-Based Access Control determines which actions an identity can perform on an Azure resource.

Our application only needs to publish messages. It does not need to retrieve or delete queue messages.

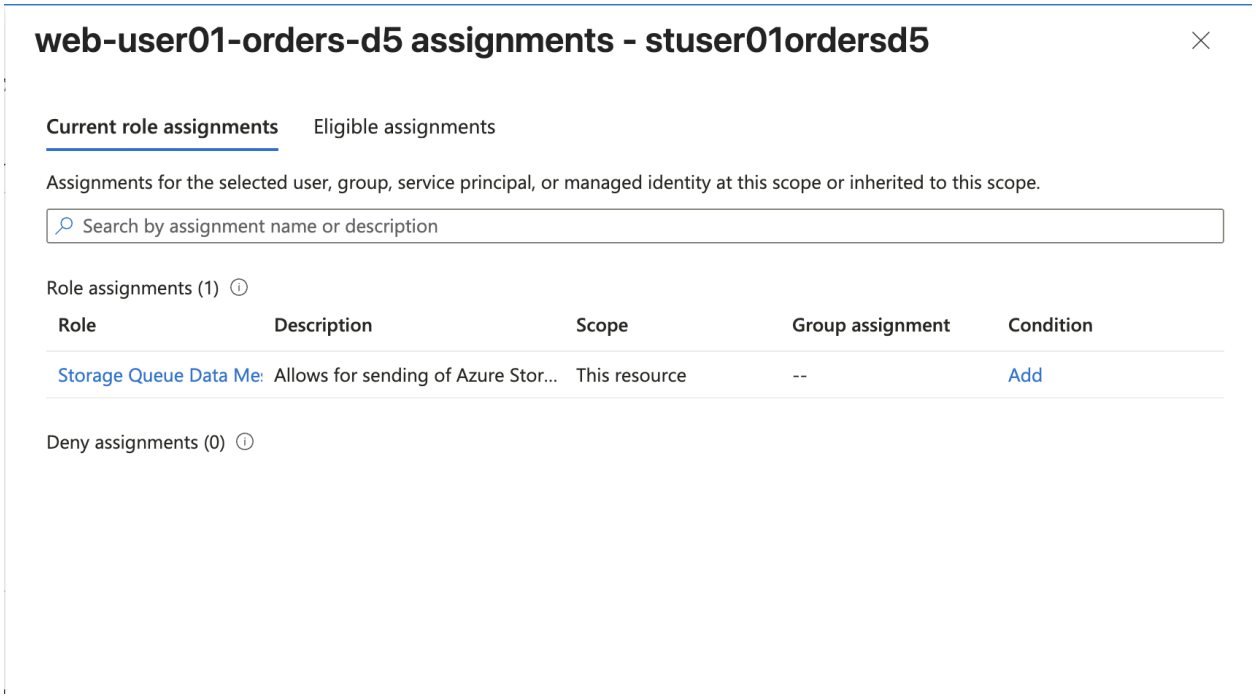
### Why are we assigning this role?

Enabling Managed Identity establishes the application's identity, but it does not automatically grant access to Storage Queue data.

We must explicitly authorize the identity.

### Execution

1. Open the Storage Account.
2. Select Access control (IAM).
3. Select Add → Add role assignment.
4. Search for and select `Storage Queue Data Message Sender`.
5. Select Next.
6. For Assign access to, choose Managed identity.
7. Select the subscription containing the Web App.
8. Choose the managed identity for `web-user01-orders-d5`.
9. Select Review + assign.




## Role understanding

| Role | Permission |
| --- | --- |
| Storage Queue Data Message Sender | Add messages to a queue. |
| Storage Queue Data Message Processor | Retrieve and delete messages. |
| Storage Queue Data Reader | Read and inspect queue data. |

We assign only the sender role to the production application.

**Important:** A normal management-plane Contributor role does not automatically provide permission to access queue messages.

**Expected outcome:** Production Web App identity has Storage Queue Data Message Sender permission on the Storage Account.

Allow time for role assignment propagation before testing access.

## Phase 4 – Integrate Azure Queue Storage with ASP.NET Core

### Concept – Application Integration

Our ASP.NET Core application currently handles web requests. We will extend it with a service responsible for publishing order messages to Azure Queue Storage.

We will use the Azure SDK for .NET rather than implementing the Storage REST API manually.

### Why these packages?

| Package | Responsibility |
| --- | --- |
| Azure.Identity | Provides Managed Identity authentication through DefaultAzureCredential. |
| Azure.Storage.Queues | Provides the QueueClient API for sending messages. |

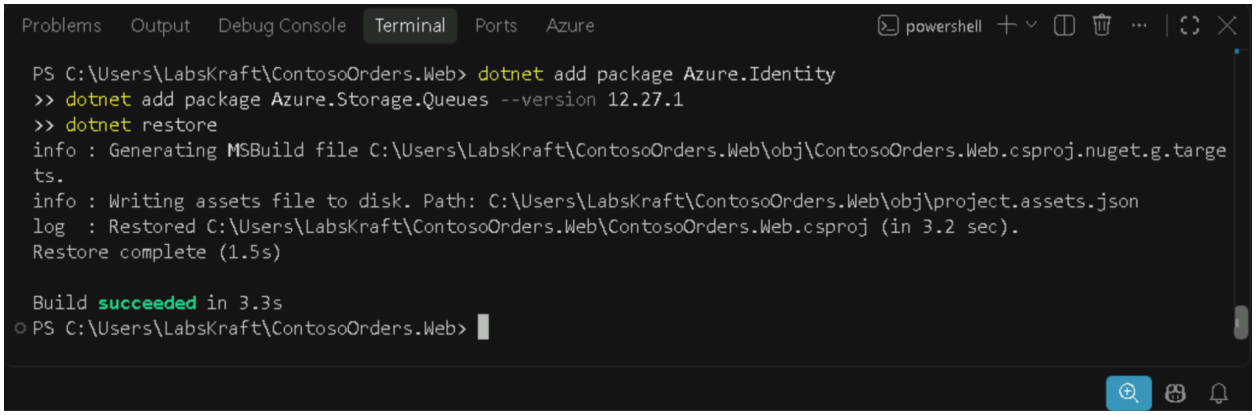
### Step 1 – Open the existing project

Open VS Code Terminal:

```
None
cd ContosoOrders.Web
code .
```

### Step 2 – Install required packages

```
None
dotnet add package Azure.Identity
dotnet add package Azure.Storage.Queues --version
12.27.1
dotnet restore
```




**Engineering understanding – Why Base64 encoding?**

We will explicitly configure Base64 message encoding.

The Azure Storage Queue SDK does not encode messages by default in this version, while Azure Functions Queue triggers commonly expect Base64-encoded messages by default.

This setting keeps the producer compatible with the planned Lab 5.3 consumer.

**Step 3 – Configure Queue Storage**

Open `appsettings.json`.

Add the following configuration:

```
None
{
  "QueueStorage": {
```

```
    "ServiceUri":
"https://stuser01ordersd5.queue.core.windows.net",
    "QueueName": "orders"
  }
}
```

Replace the Storage Account name with your actual name.

**Important:**

- ServiceUri is the Queue Storage service endpoint.
- QueueName identifies the destination queue.
- This is not a connection string.
- Do not add storage account keys or connection strings.

**Step 4 – Create the Queue Service**

Create a folder named `Services`.

Inside it, create:

`Services/QueueOrderService.cs`

Add:

```
None
using Azure.Identity;
using Azure.Storage.Queues;
using Azure.Storage.Queues.Models;
using System.Text.Json;

namespace ContosoOrders.Web.Services;
```

```
public class QueueOrderService
{
    private readonly QueueClient _queueClient;

    public QueueOrderService(IConfiguration
configuration)
    {
        string serviceUri =
            configuration["QueueStorage:ServiceUri"]
            ?? throw new
InvalidOperationException("Queue service URI is
missing.");

        string queueName =
            configuration["QueueStorage:QueueName"]
            ?? throw new
InvalidOperationException("Queue name is missing.");

        var options = new QueueClientOptions
        {
            MessageEncoding =
QueueMessageEncoding.Base64
        };

        _queueClient = new QueueClient(
            new
Uri($"{serviceUri.TrimEnd('/')}/{queueName}"),
            new DefaultAzureCredential(),
```

```
            options);
    }

    public async Task<string> SendOrderAsync(object
order)
    {
        string message =
JsonSerializer.Serialize(order);

        await _queueClient.SendMessageAsync(message);

        return message;
    }
}
```

**Understand the implementation**

| Code component | Responsibility |
| --- | --- |
| QueueClient | Communicates with Azure Queue Storage. |
| DefaultAzureCredential | Obtains an authentication credential. |
| QueueClientOptions | Configures queue message encoding. |
| JsonSerializer | Converts the order object into JSON. |

| | |
|---|---|
| SendMessageAsync | Publishes the message to Queue Storage. |

**Behind the scenes**

When `SendMessageAsync` executes:

1. The order object is serialized into JSON.
2. The QueueClient uses DefaultAzureCredential.
3. In Azure App Service, the credential chain uses the available Managed Identity.
4. A token is obtained for Azure Storage.
5. Azure Storage validates the identity and RBAC permissions.
6. The message is published to the `orders` queue.

**Important:** DefaultAzureCredential can use different credentials in development and Azure hosting environments. The production application uses its system-assigned Managed Identity.

**Step 5 – Register the Service in Program.cs**

Open `Program.cs`.

Add the namespace at the top of the file:

```
None
using ContosoOrders.Web.Services;
```

Register the service before:

```
None
var app = builder.Build();
```

Add:

```
None
builder.Services.AddSingleton<QueueOrderService>();
```

**Why are we registering the service?**

Dependency Injection allows ASP.NET Core to provide QueueOrderService to application components that require it, without manually constructing the service in every page.

**Step 6 – Create the Order Submission Page**

Inside the `Pages` folder, create:

`Pages/SubmitOrder.cshtml`

Add:

```
None
@page
@model ContosoOrders.Web.Pages.SubmitOrderModel

<h1>Submit Contoso Order</h1>

<form method="post">
    <label>Customer Name</label>
    <input asp-for="CustomerName" required />

    <label>Product</label>
    <input asp-for="Product" required />

    <label>Quantity</label>
```

```
    <input asp-for="Quantity" type="number" min="1"
required />

    <button type="submit">Submit Order</button>
</form>

@if (!string.IsNullOrEmpty(Model.StatusMessage))
{
    <p>@Model.StatusMessage</p>
}
```

**Purpose:** This page provides a simple user interface for entering order information.

**Step 7 – Create the Page Model**

Inside the `Pages` folder, create:

`Pages/SubmitOrder.cshtml.cs`

Add:

```
None
using ContosoOrders.Web.Services;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace ContosoOrders.Web.Pages;

public class SubmitOrderModel : PageModel
{
```

```
    private readonly QueueOrderService _queueService;

    public SubmitOrderModel(QueueOrderService
queueService)
    {
        _queueService = queueService;
    }

    [BindProperty]
    public string CustomerName { get; set; } = "";

    [BindProperty]
    public string Product { get; set; } = "";

    [BindProperty]
    public int Quantity { get; set; }

    public string StatusMessage { get; set; } = "";

    public async Task<IActionResult> OnPostAsync()
    {
        if (string.IsNullOrWhiteSpace(CustomerName)
||
            string.IsNullOrWhiteSpace(Product) ||
            Quantity < 1)
        {
            ModelState.AddModelError("", "Enter valid
order details.");
            return Page();
```

```
        }

        var order = new
        {
            OrderId =
$"ORD-{Guid.NewGuid():N}"[..12].ToUpperInvariant(),
            CustomerName,
            Product,
            Quantity,
            SubmittedAtUtc = DateTime.UtcNow
        };

        try
        {
            await
_queueService.SendOrderAsync(order);
            StatusMessage = $"Order {order.OrderId}
sent to the queue.";
        }
        catch (Exception ex)
        {
            // Keep the detailed exception in server
logs, not in the browser.
            Console.Error.WriteLine(ex);
            StatusMessage = "Order submission failed.
Check application logs.";
        }

        return Page();
```

```
    }
}
```

**Understand the order submission flow**

1. The customer enters order information.
2. Razor Pages receives the form submission.
3. The page model validates the input.
4. An order object is created with a unique OrderId.
5. QueueOrderService serializes the order.
6. The message is sent to Azure Queue Storage.
7. The user receives a submission confirmation.

**Important integration contract for Lab 5.3**

The message contains:

| Property | Purpose |
|---|---|
| OrderId | Unique order identifier. |
| CustomerName | Customer submitting the order. |
| Product | Ordered product. |
| Quantity | Requested quantity. |
| SubmittedAtUtc | Submission timestamp in UTC. |

Lab 5.3 must consume this JSON structure. Do not rename these properties without updating the consumer accordingly.

**Step 8 – Add Navigation to the Order Page**

Open:

`Pages/Shared/_Layout.cshtml`

Add the following link inside the existing navigation list:

```
None
<a class="nav-link text-dark"
   asp-page="/SubmitOrder">Submit Order</a>
```

This allows users to open the order submission page through the application navigation.

**Step 9 – Build and Validate**

Execute:

```
None
dotnet restore
dotnet build
```




**Expected result:**

- Dependencies restore successfully.
- Build completes with zero errors.
- Warnings are reviewed rather than ignored.

**Expected application changes:**

The project now contains:

- QueueOrderService.cs
- SubmitOrder.cshtml
- SubmitOrder.cshtml.cs
- Updated Program.cs
- Updated navigation
- Queue Storage configuration

## Phase 5 – Configure Production Application Settings

**Concept – Environment-specific Configuration**

In enterprise environments, the same application may be deployed across multiple environments.

We keep the application code independent of environment-specific endpoint values.

ASP.NET Core supports environment-variable configuration using double underscores to represent nested configuration keys.

For example:

`QueueStorage__ServiceUri` maps to `QueueStorage:ServiceUri`.

**Execution**

Open the production Web App:

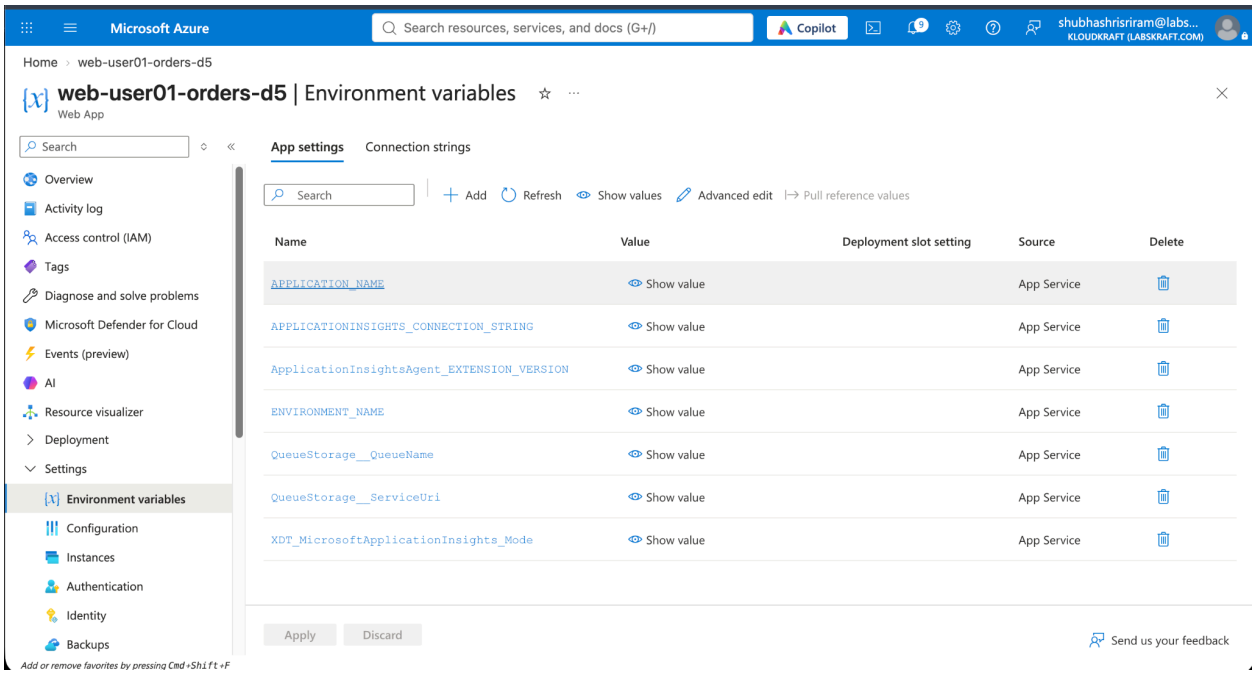
`web-user01-orders-d5`

Navigate to:

Settings → Environment variables (or Configuration, depending on the portal view).

Under App settings, add:

| Name | Value |
| --- | --- |
| QueueStorage__ServiceUri | https://stuser01ordersd5.queue.core.windows.net |
| QueueStorage__QueueName | orders |




Replace the Storage Account name with your actual name.

Save the settings and allow the Web App to restart.

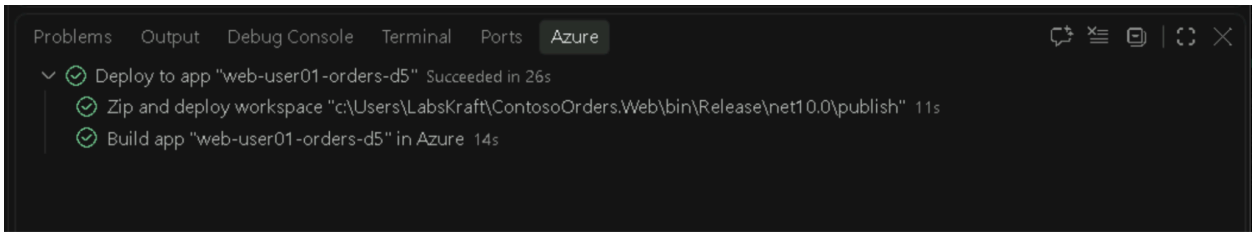
**Expected outcome:** Production Web App has the correct Queue Storage endpoint and queue name configured.

## Phase 6 – Deploy and Test Production

**Purpose:** Publish the updated application and verify the complete web-to-queue integration.

### Execution

1. Open VS Code.
2. Sign in to Azure through the Azure App Service extension.
3. Expand App Service in the Azure panel.
4. Right-click `web-user01-orders-d5`.
5. Select Deploy to Web App.
6. Select the `ContosoOrders.Web` project folder.
7. Confirm deployment.
8. Wait for deployment success.
9. Open the production Web App URL.
10. Navigate to Submit Order.

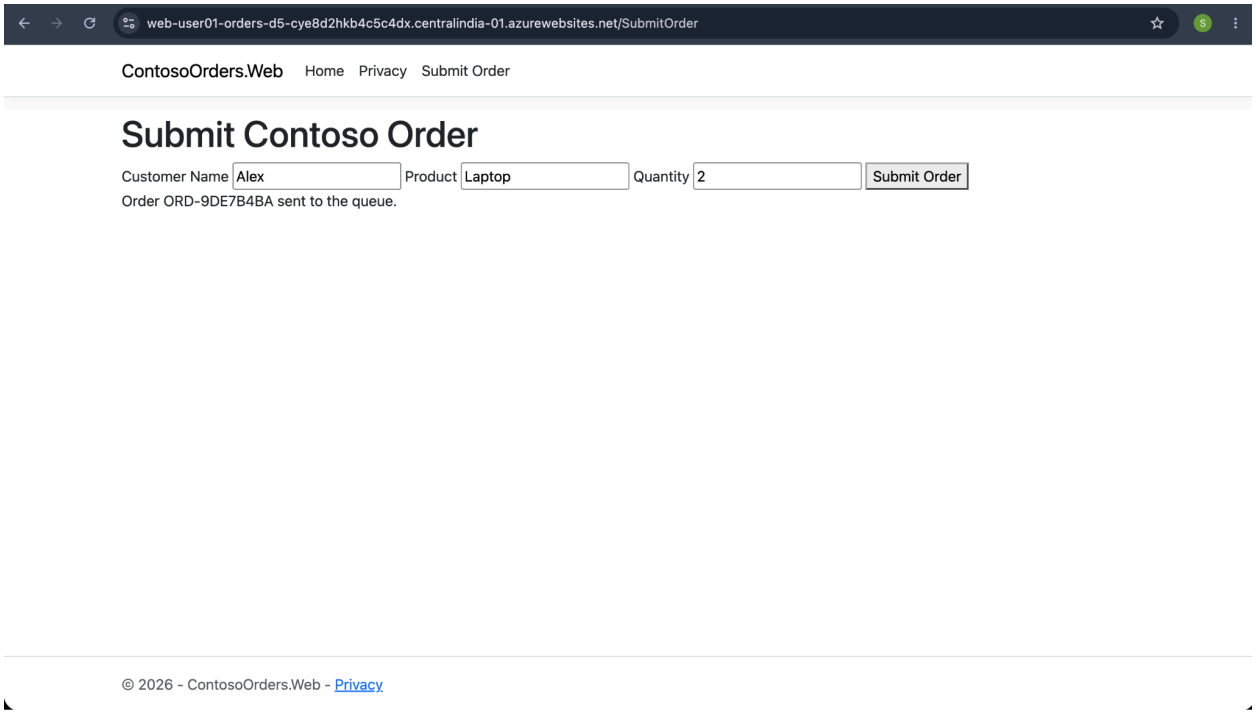


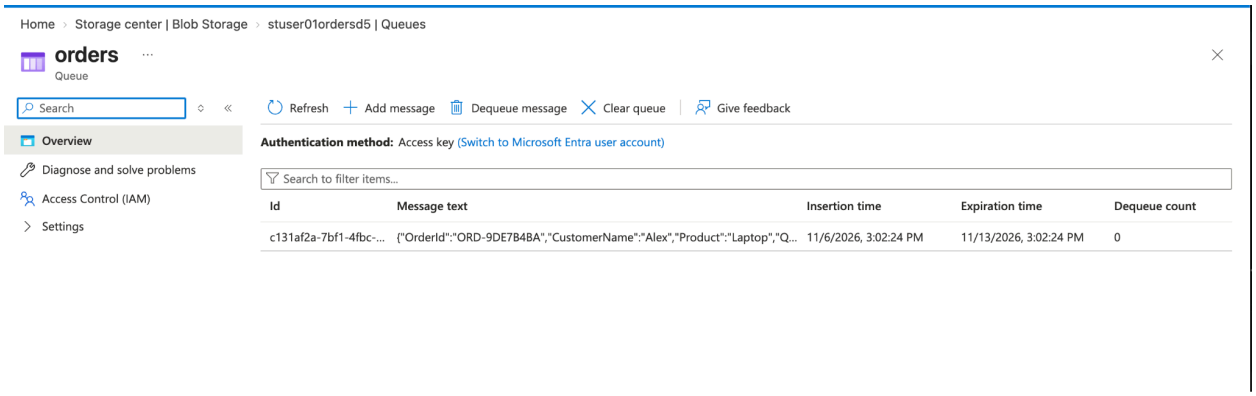

Submit a test order:

| Field | Test value |
|---|---|
| Customer Name | Alex |
| Product | Laptop |
| Quantity | 2 |

Now verify the message:

1. Open the Storage Account.
2. Navigate to Data storage → Queues.
3. Open the `orders` queue.
4. Inspect its messages using the portal's available queue message view.




## Expected result

- Application confirms successful order submission.
- The `orders` queue contains a message with the submitted order details.

## Engineering understanding

The web application is now acting as a queue producer.

The application does not need to know how the background processing system works. It only needs to publish the message successfully.

**Troubleshooting:** If the message does not appear, inspect the Web App Log Stream, the Queue Storage endpoint and the production identity's role assignment before modifying application code.

**Portal access note:** The application's sender identity cannot read queue messages. If the signed-in participant uses Microsoft Entra authentication to inspect messages in the portal, that user needs appropriate data-reader and resource-navigation permissions. Do not grant those permissions to the application's managed identity.

## Phase 7 – Create the Staging Deployment Slot

### Concept – Why do we need staging?

A staging slot provides a separate environment for testing application updates before they are promoted to production.

This reduces the risk of deploying untested changes directly to the live application.

### Why are we reusing the existing App Service Plan?

The Standard S1 plan from Lab 5.1 supports deployment slots. Both production and staging can use the same App Service Plan, so another plan is unnecessary.

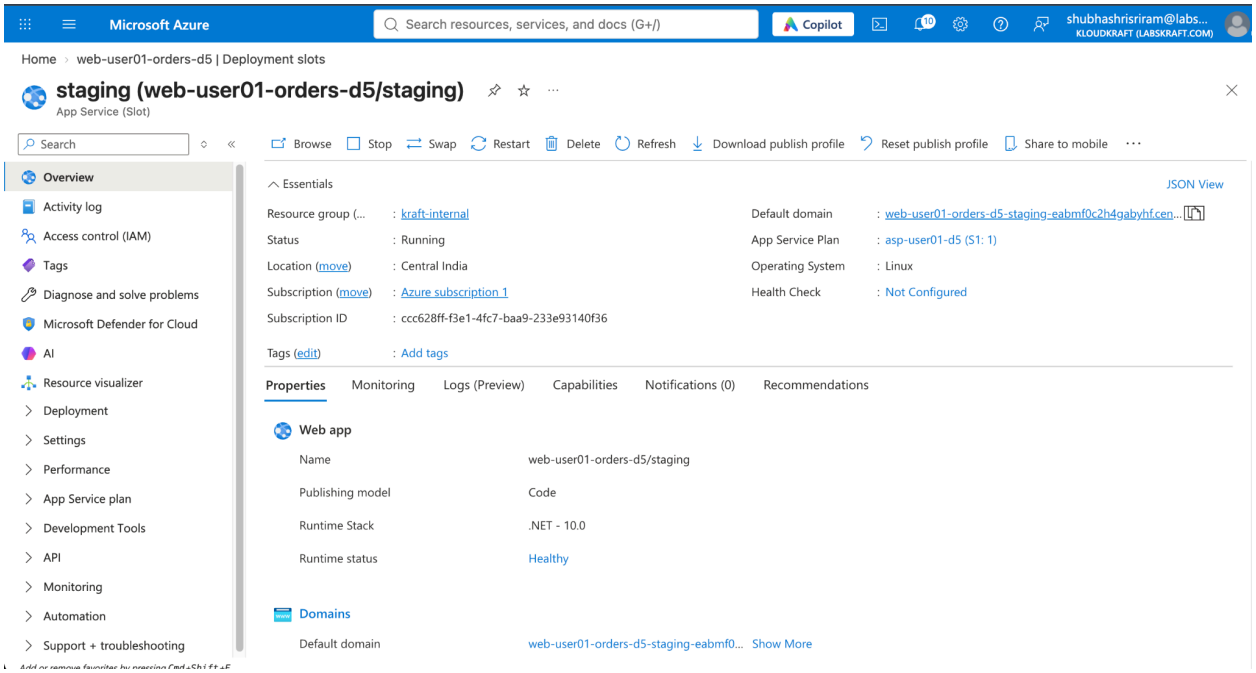
### Execution

1. Open the production Web App.
2. Under Deployment, select Deployment slots.
3. Select Add Slot.
4. Enter `staging`.
5. Select Clone settings from production.
6. Select Add.
7. Wait for staging slot provisioning to complete.

Open the staging slot and copy its URL.

## Example:

```
https://web-user01-orders-d5-staging.azurewebsites.net
```




## Engineering understanding

Production and staging have separate application endpoints and deployment environments, but share the underlying App Service Plan capacity.

**Expected outcome:** The staging deployment slot is created and accessible.

## Phase 8 – Enable Managed Identity on Staging

**Important concept**

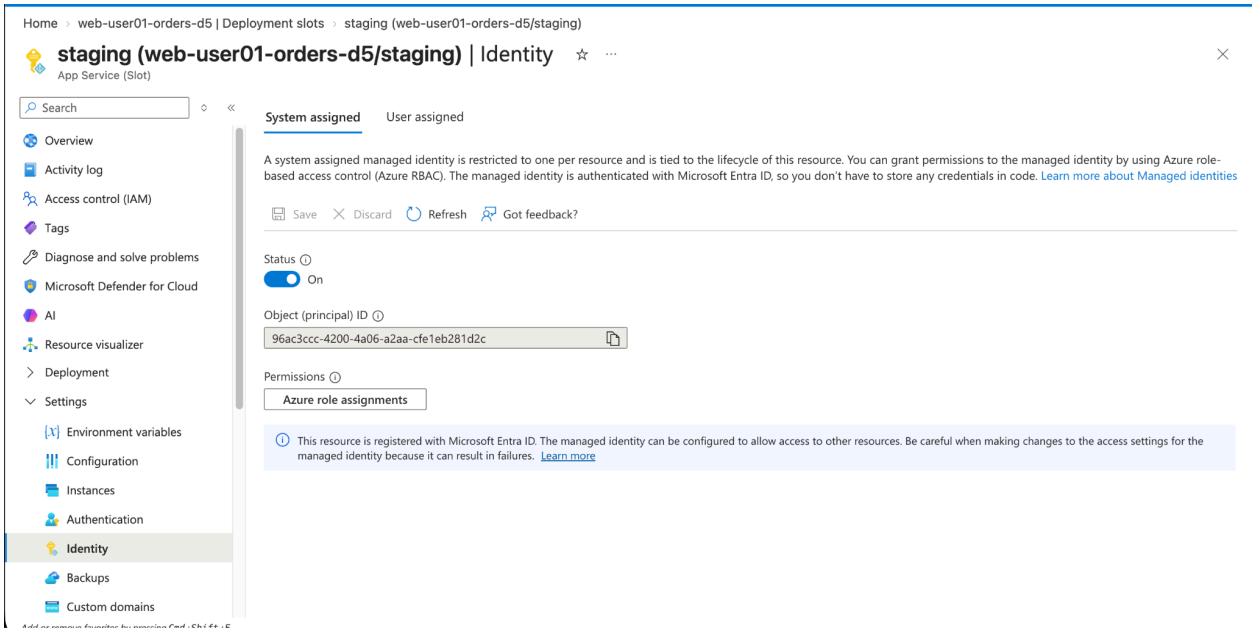
A deployment slot has its own Managed Identity configuration. The production identity is not automatically reused by staging.

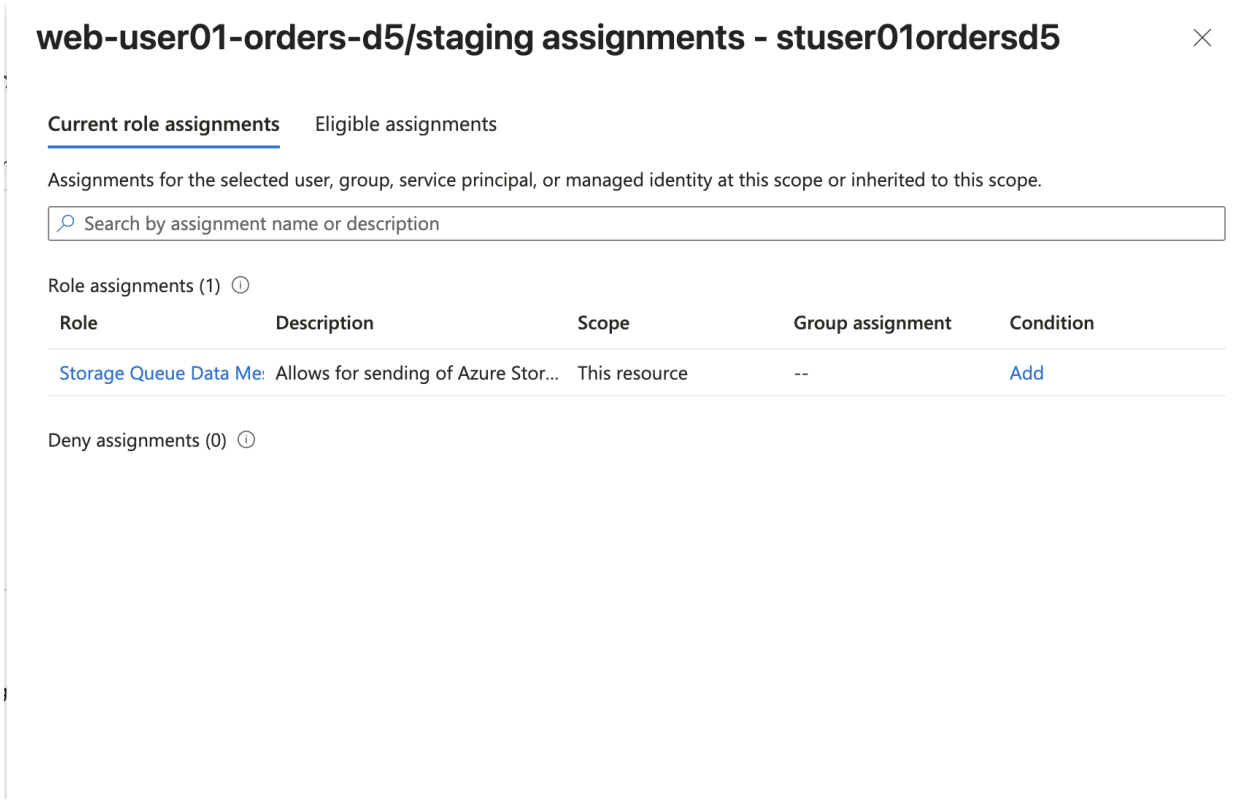
**Why are we enabling another identity?**

Both applications need to communicate with the same Storage Account. Each slot must authenticate independently.

**Execution**

1. Open the `staging` slot from the Web App's deployment slots page.
2. Navigate to Settings → Identity.
3. Enable System assigned.
4. Save the configuration.
5. Record the staging identity's Object (principal) ID.
6. Return to the Storage Account.
7. Select Access control (IAM).
8. Select Add → Add role assignment.
9. Choose `Storage Queue Data Message Sender`.
10. Select Managed identity.
11. Select the staging slot's identity.
12. Select Review + assign.




## Expected outcome

Both production and staging identities have permission to send messages to the `orders` queue.

**Engineering understanding**

The two identities are independent security principals:

- Production identity authenticates production requests.
- Staging identity authenticates staging requests.

This ensures that each environment has its own access permissions.

## Phase 9 – Configure Staging Application Settings

**Purpose:** Provide the staging application with the same Queue Storage destination as production.

**Execution**

Open the staging slot, not the production Web App.

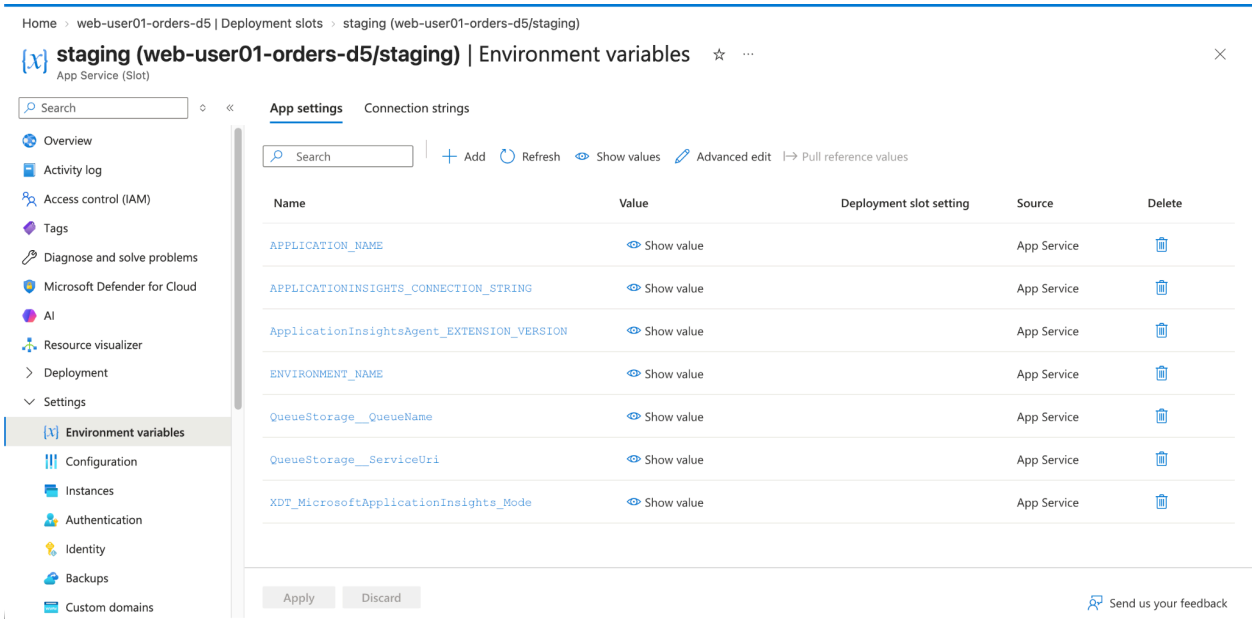
Navigate to:

Settings → Environment variables.

Under App settings, add:

| Name | Value |
|---|---|
| QueueStorage__ServiceUri | https://stuser01ordersd5.queue.core.windows.net |
| QueueStorage__QueueName | orders |

Save the settings and restart if prompted.




### Important configuration decision

Keep the queue endpoint and queue name consistent between production and staging.

These two settings can remain ordinary application settings because both slots use the same queue destination.

Do not mark them as deployment slot settings unless you specifically intend them to remain attached to their respective slots.

The environment marker introduced later will be slot-specific.

**Expected outcome:** Staging has the configuration required to communicate with the `orders` queue.

## Phase 10 – Deploy and Test the Staging Version

**Purpose:** Validate a new application version in staging before promoting it to production.

For a clear demonstration, make a small visible change to the application.

Open:

`Pages/SubmitOrder.cshtml`

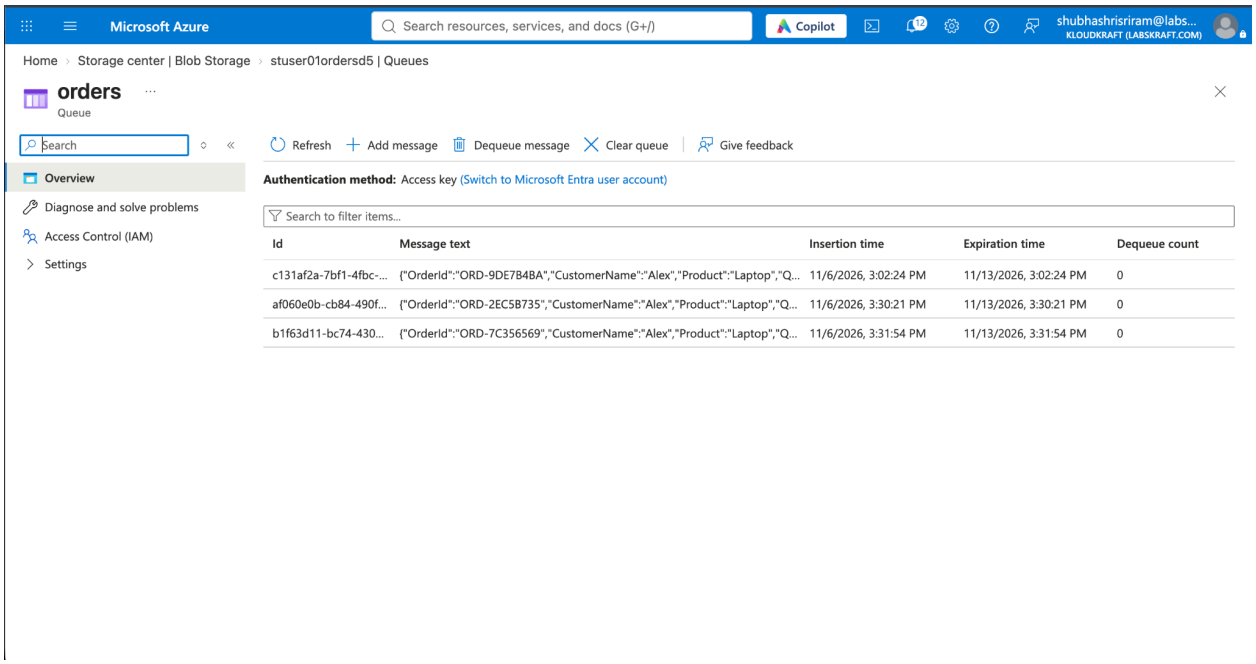
Temporarily add:

```
None
<p>Application Version: STAGING</p>
```

**Execution**

1. Open VS Code.
2. Expand the Azure App Service panel.
3. Right-click the `staging` slot.
4. Select Deploy to Web App.
5. Choose the `ContosoOrders.Web` project folder.
6. Confirm deployment.
7. Open the staging URL.
8. Verify the staging marker appears.
9. Navigate to Submit Order.
10. Submit a test order.
11. Verify that the staging application successfully publishes a message to the same `orders` queue.




## Expected result

- Staging application is accessible.
- Staging marker is visible.
- Order submission works.
- A message appears in the `orders` queue.
- Staging uses its own Managed Identity.

## Engineering understanding

We have validated the new application version without directly changing the production application.

Both slots can independently send messages because both identities have the required permissions.

## Phase 11 – Configure Environment Markers and Swap Slots

### Concept – What is a Slot Swap?

A slot swap exchanges the application content and swappable configuration between two deployment slots.

The staging application is promoted into production, while the previous production application moves into staging.

### Why are we doing this?

Organizations use slot swaps to release tested application versions into production through a controlled deployment process.

#### Step 1 – Remove the Temporary Staging Marker

Remove the temporary hardcoded line from `SubmitOrder.cshtml`:

```
None
<p>Application Version: STAGING</p>
```

We will replace it with an environment-specific marker.

#### Step 2 – Configure Environment-specific Settings

Open the production Web App.

Navigate to:

Settings → Environment variables.

Add:

| Name | Value |
| --- | --- |
| APP_ENVIRONMENT | PRODUCTION |

Mark `APP_ENVIRONMENT` as a Deployment slot setting.

Save the changes.

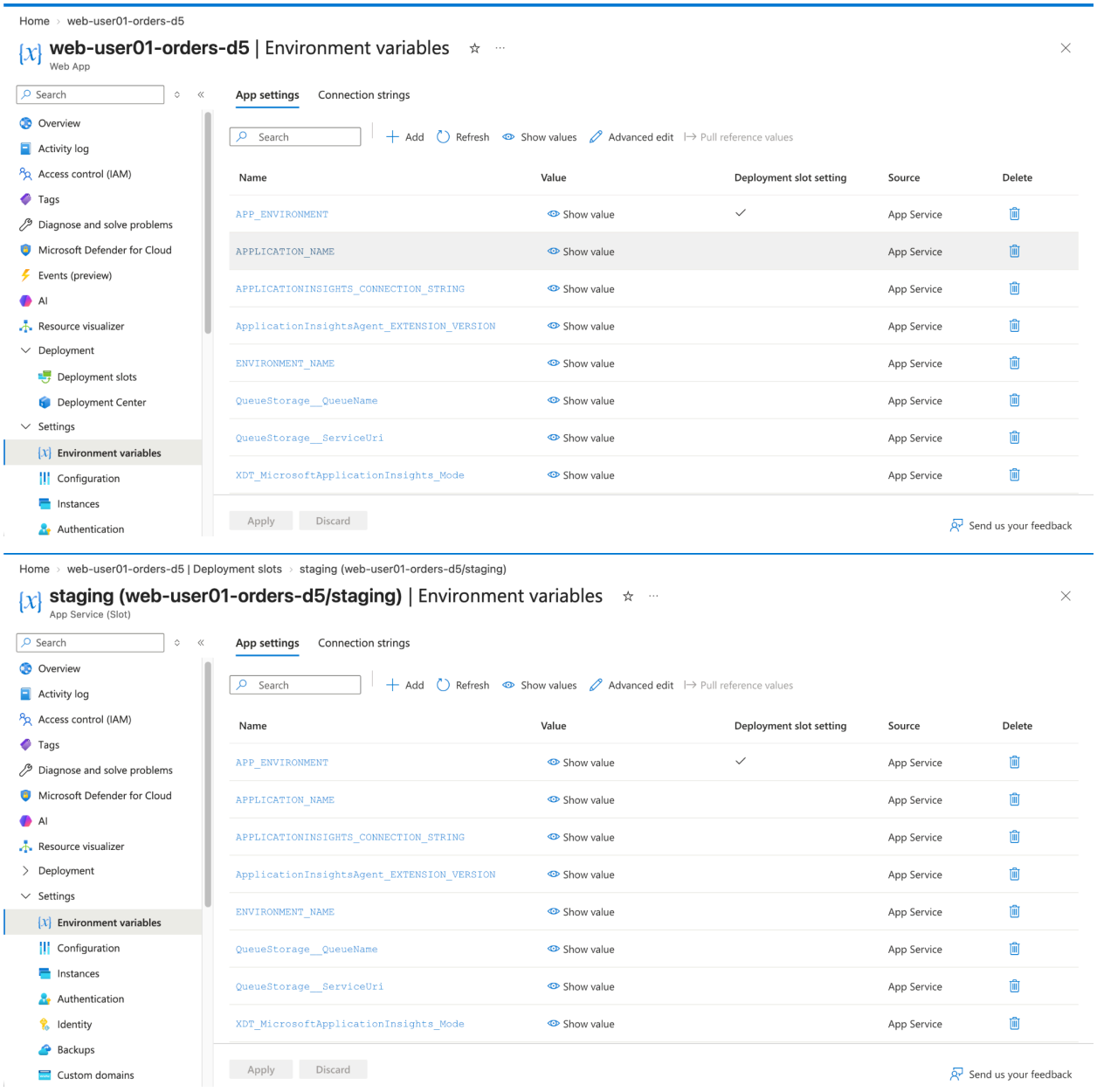
Now open the staging slot.

Add:

| Name | Value |
| --- | --- |
| APP_ENVIRONMENT | STAGING |

Mark this setting as a Deployment slot setting as well.

Save the changes.







## Step 3 – Understand Slot-specific Settings

Deployment slot settings remain attached to their respective slots during a swap.

This is important because the application should continue identifying its environment correctly after deployment.

For example:

- Production continues to report PRODUCTION.
- Staging continues to report STAGING.

The Managed Identities also remain associated with their respective slots.

**Step 4 – Update the Application Marker**

Open:

`Pages/SubmitOrder.cshtml`

Add:

```
None
<p>
    Environment:

@System.Environment.GetEnvironmentVariable("APP_ENVIR
ONMENT")
</p>
```

Deploy this updated application to the staging slot using VS Code.

Open the staging URL and verify:

```
None
Environment: STAGING
```

**Expected result:** The staging application displays the correct environment marker.

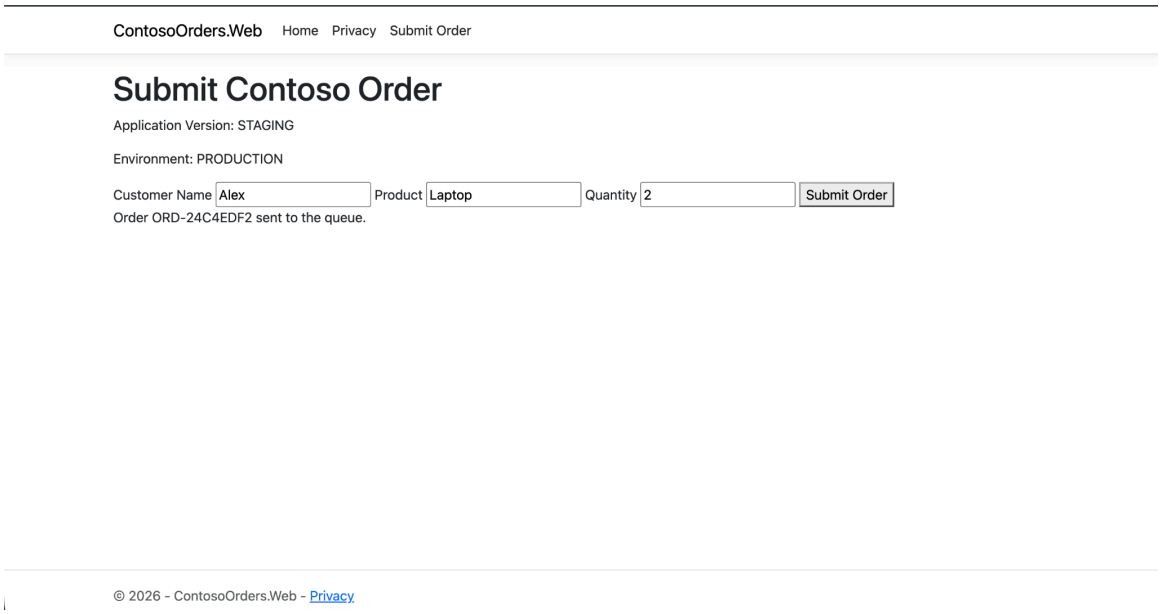
**Step 5 – Perform the Slot Swap**

Before swapping, verify:

- Staging application is running.
- Staging can submit orders.
- Queue messages are being published successfully.
- Production is healthy.
- Environment-specific settings are configured correctly.

Now:

1. Open the production Web App.
2. Navigate to Deployment slots.
3. Select Swap.
4. Set Source to `staging`.
5. Set Target to `Production`.
6. Review the configuration changes.
7. Select Start Swap.
8. Wait for the operation to complete.
9. Open the production URL.
10. Verify the updated application is live.
11. Open the staging URL.
12. Verify the previous production application is now there.




## Expected result

The new application version is serving production traffic.

The environment-specific marker remains:

| Slot | Expected marker |
|---|---|
| Production | PRODUCTION |
| Staging | STAGING |

**Important:** Managed Identities are not swapped. Production and staging continue using their respective identities and role assignments.

## Phase 12 – Final Validation

Perform the following checks:

| Validation | Expected result |
|---|---|
| Production Web App | Running |
| Staging slot | Running |
| Production Managed Identity | Enabled |
| Staging Managed Identity | Enabled |
| Production queue role | Storage Queue Data Message Sender |
| Staging queue role | Storage Queue Data Message Sender |
| Production order submission | Successful |
| Staging order submission | Successful |

| | |
|---|---|
| Queue `orders` | Contains submitted messages |
| Storage authentication | Managed Identity |
| Account keys / connection strings in application | Not used |
| Deployment slot swap | Successful |
| Production environment marker | PRODUCTION |
| Staging environment marker | STAGING |

## 6. Troubleshooting Guide

| Symptom | What to investigate |
|---|---|
| HTTP 403 / AuthorizationPermissionMismatch | Correct identity and Storage Queue Data Message Sender role assignment. |
| Queue not found | Queue name and Storage Account service URI. |
| Authentication failure | Managed Identity configuration on the slot handling the request. |
| Works in production, fails in staging | Staging identity and separate RBAC assignment. |
| Configuration missing | QueueStorage__ServiceUri and QueueStorage__QueueName settings. |
| Message unreadable by consumer | Base64 encoding and consumer configuration. |

| | |
|---|---|
| Changes not visible | Deployment status, restart status and correct slot URL. |
| Slot swap fails | Review swap operation details, application startup and slot configuration. |
| Queue submission fails after role assignment | Verify identity, role scope and allow time for RBAC propagation. |

**Operational troubleshooting approach**

When a queue submission fails:

1. Verify the application is running.
2. Check the configured Storage Account endpoint.
3. Confirm the correct Managed Identity is enabled.
4. Verify its RBAC assignment.
5. Inspect App Service Log Stream.
6. Check the application exception details in server logs.
7. Retry after resolving the actual issue.

Role assignment changes can take up to approximately 10 minutes to propagate. Avoid repeatedly changing permissions while waiting for propagation.

## 7. Evidence Collection

Capture the following screenshots:

- Storage Account showing the `orders` queue.
- Production Web App showing enabled Managed Identity.
- Production identity IAM role assignment.
- Staging slot showing enabled Managed Identity.
- Staging identity IAM role assignment.

- Successful production order submission.
- Production queue message visible.
- Staging slot showing the staging version.
- Successful staging order submission.
- Staging queue message visible.
- Deployment slot swap completed.
- Production URL showing PRODUCTION.
- Staging URL showing STAGING.

## 8. Production Engineering Takeaways

At the end of this lab, participants should understand:

- Why asynchronous messaging is used in enterprise applications.
- How Azure Queue Storage decouples application components.
- How a web application publishes messages using the Azure SDK.
- How Managed Identity authenticates without embedded credentials.
- Why Azure RBAC is required even after enabling Managed Identity.
- Why production and staging require separate identities and permissions.
- How deployment slots support controlled application releases.
- How to troubleshoot authentication, authorization and message-delivery issues.
- How message encoding and message structure affect downstream consumers.

**Real-world corporate scenario**

An e-commerce company receives thousands of customer orders. Instead of making the website wait for inventory checks, notifications and downstream order processing, the application publishes each order to a queue.

A separate processing service consumes the messages and performs the required operations.

If the processing service is temporarily unavailable, submitted messages remain available in the queue for later processing, subject to queue retention and message lifecycle behavior.

This approach reduces direct dependency between the customer-facing application and the background processing system.

## 9. Connection to Lab 5.3

In this lab, our ASP.NET Core application acts as the message producer.

The next requirement is to process the messages automatically.

In Lab 5.3, we will introduce Azure Functions using the .NET 10 isolated worker model.

The Function will:

- Listen for messages arriving in the `orders` queue.
- Authenticate to Azure Storage using Managed Identity.
- Deserialize the order JSON.
- Process order information.
- Store processed output in Azure Blob Storage.
- Demonstrate failure handling and poison-message handling.

**Integration contract:** The next lab must use the same Storage Account, queue name, Base64 message encoding and order JSON structure established here.

## 10. Cleanup

Do not delete the Day 5 resources after Lab 5.2.

Lab 5.3 will reuse:

- ASP.NET Core Web App.
- App Service Plan.
- Storage Account.
- `orders` queue.
- Production deployment slot.
- Staging deployment slot.

Both Managed Identities and their role assignments must also remain available for the subsequent lab.